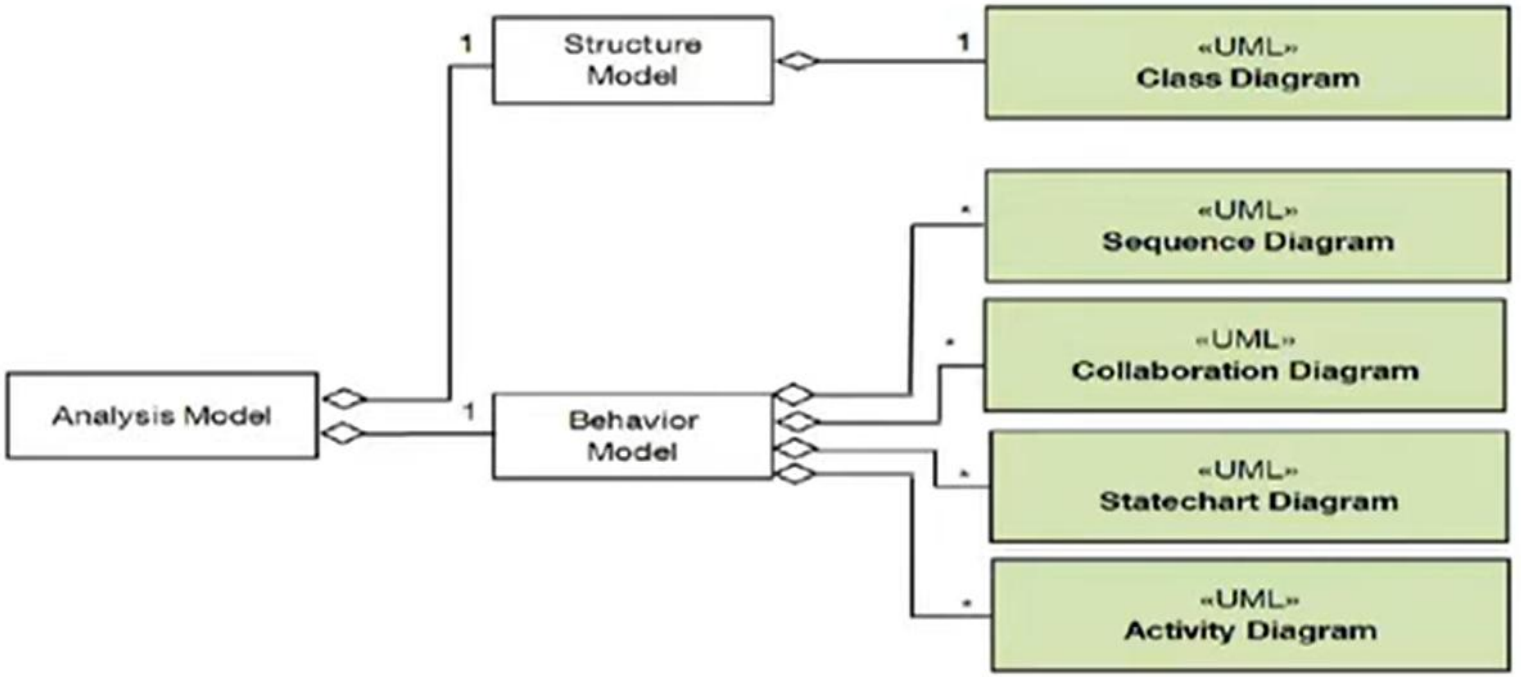
Object Oriented Analysis and Design

Communication/Collaboration Diagram

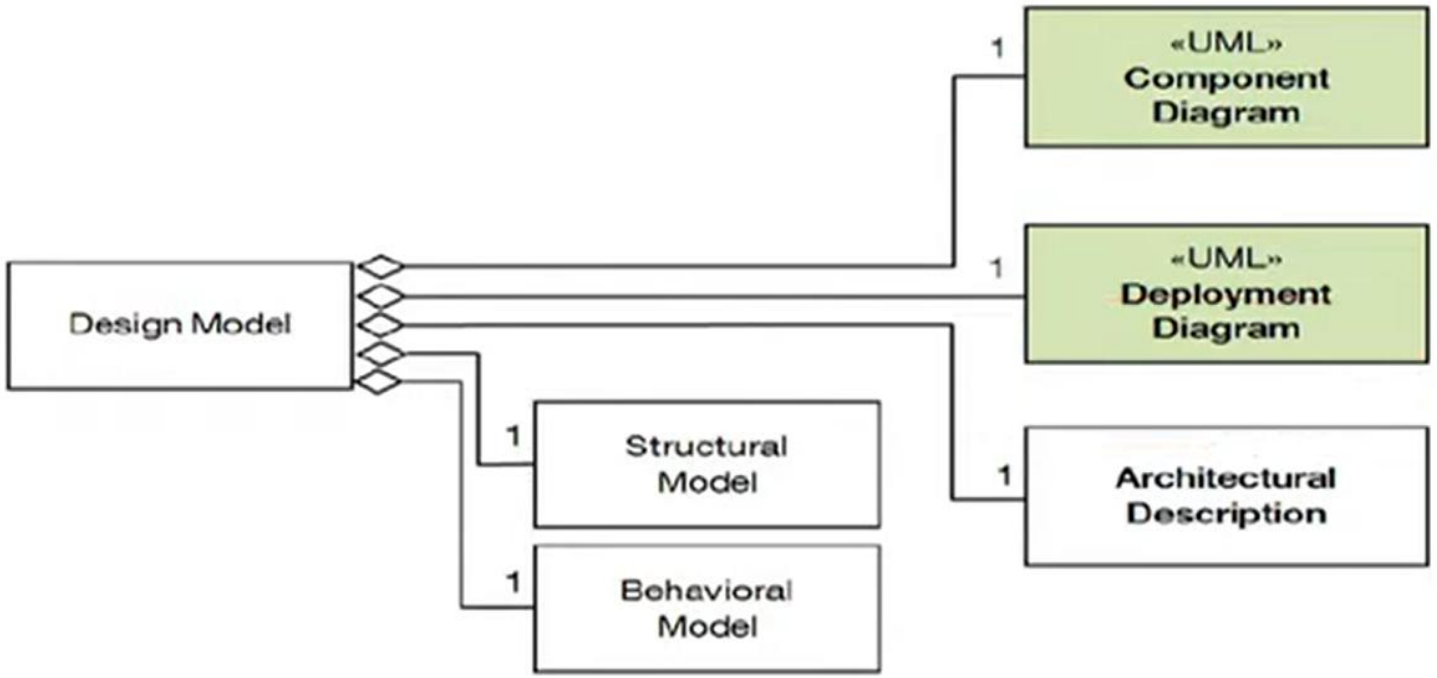
Communication (Collaboration) in SDLC Phases

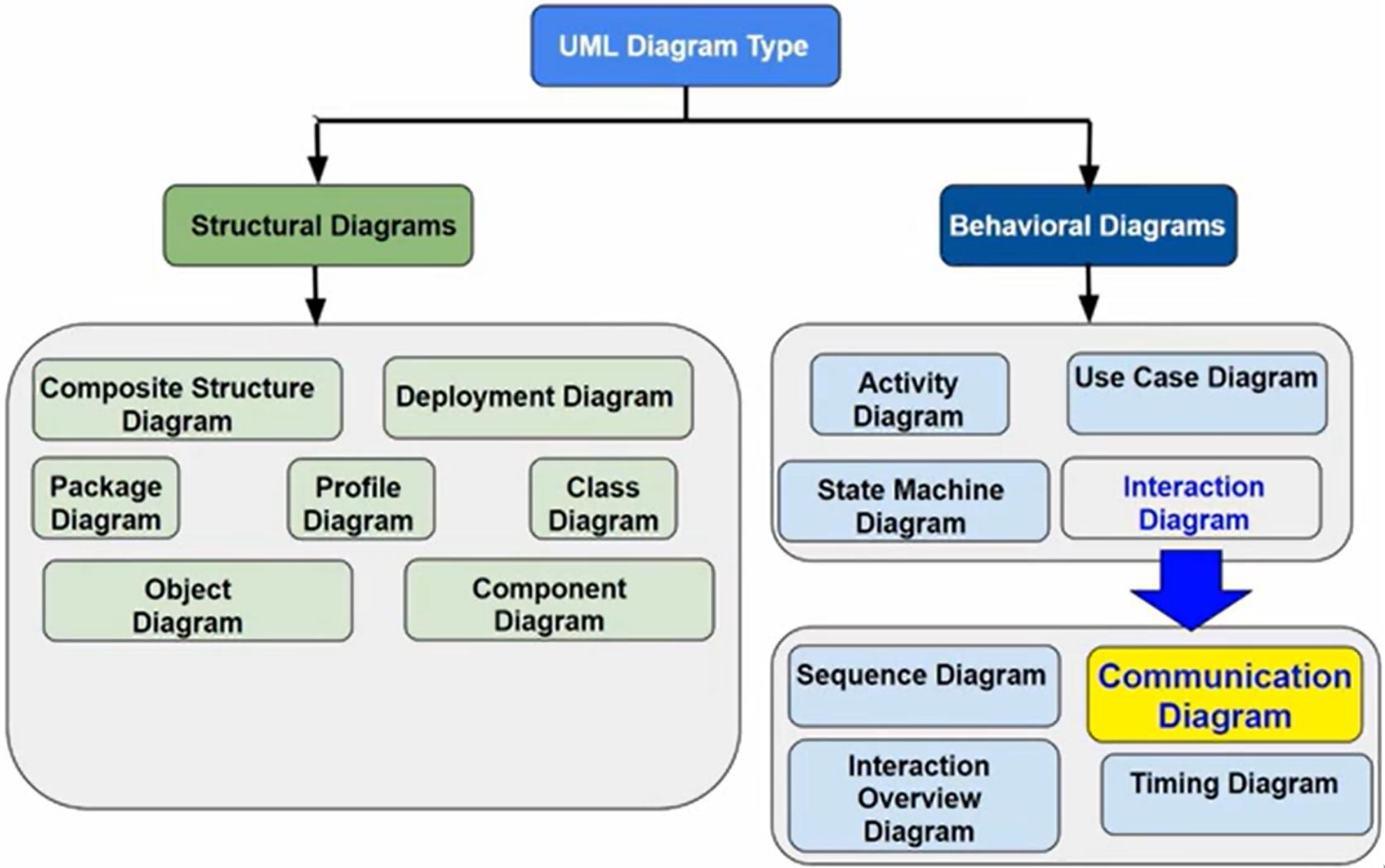
Communication (Collaboration) diagrams is a result of the analysis phase.



Communication (collaboration) diagrams in SDLC Phases

Communication (Collaboration) diagram is included in the Behavioral Model



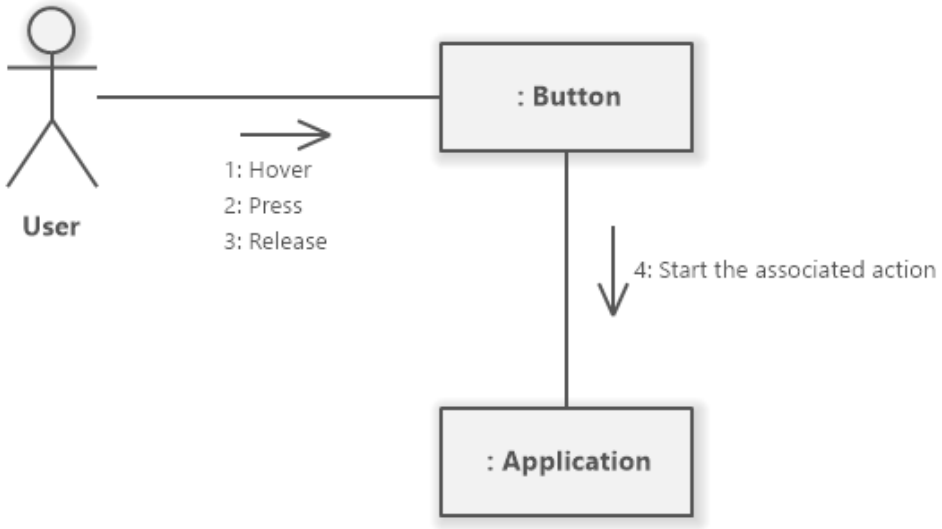

UML Diagram Type
Structural Diagrams
Behavioral Diagrams
Composite Structure Diagram
Deployment Diagram
Package Diagram
Profile Diagram
Class Diagram
Object Diagram
Component Diagram
Activity Diagram
Use Case Diagram
State Machine Diagram
Interaction Diagram
Sequence Diagram
Communication Diagram
Interaction Overview Diagram
Timing Diagram

What problem do Communication Diagrams Solve?

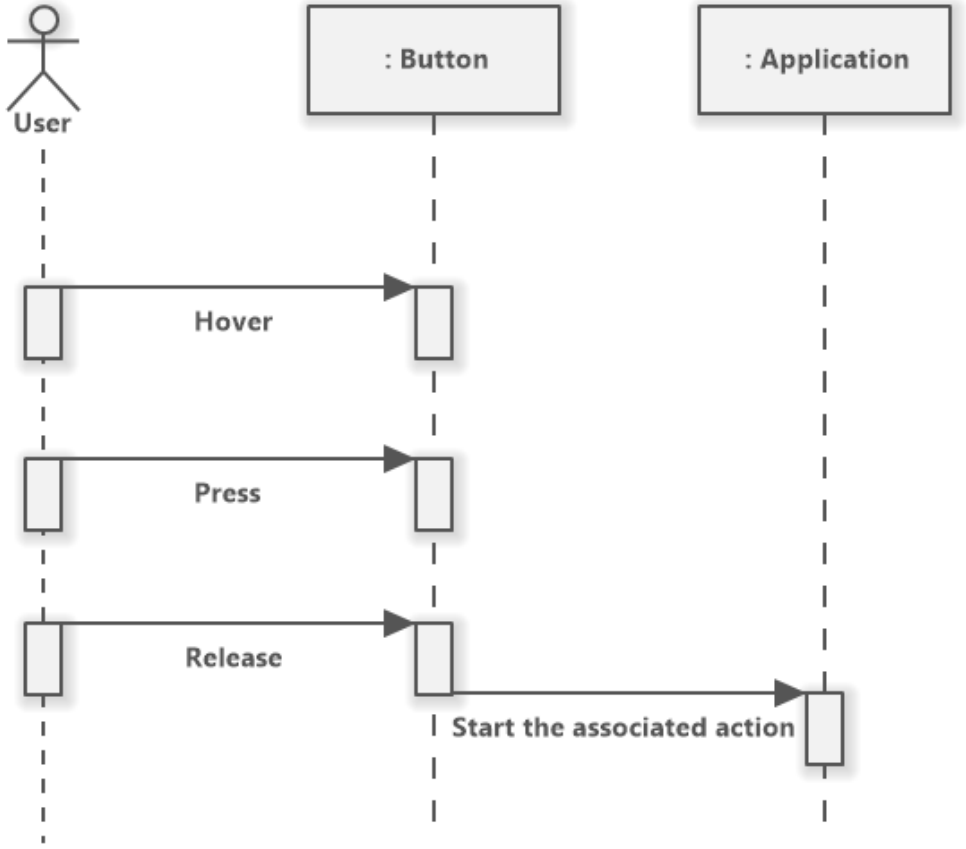
- Scenario: Imagine you're designing a smart home system where a motion sensor triggers lights, which then notify a security system, which logs the event. How would you show these interactions focusing on the RELATIONSHIPS between objects rather than TIME?



Communication vs Sequence diagram



VS.



While both show interactions, sequence diagrams focus on time order, whereas communication diagrams emphasize the structural relationships between objects and how they are linked.



What are communication diagrams

Communication diagrams emphasize object relationships and spatial layout

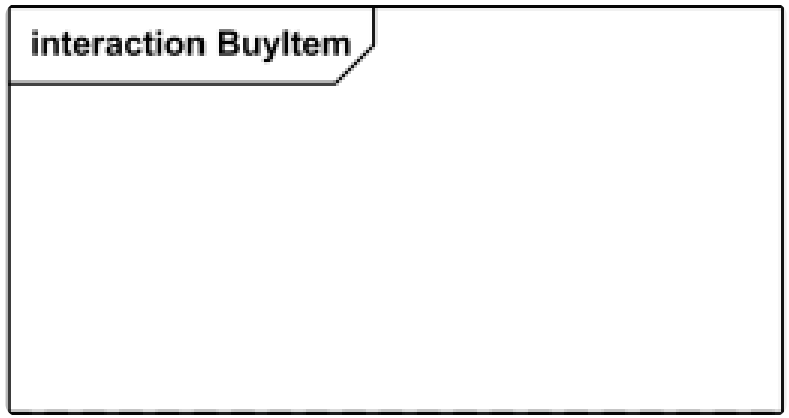
Part of UML 2.x Behavioral Diagrams (renamed from Collaboration Diagrams in UML 1.x)

Created during the Analysis Phase of SDLC

Show the Structural Organization of objects that interact.

Anatomy of Communication Diagrams

- Three Essential Components
  - Frame (optional but recommended)
  - Lifelines (The Actors)
  - Messages (The Communication)



Interaction ***Frame*** *for* ***Communication Diagram*** *BuyItem*

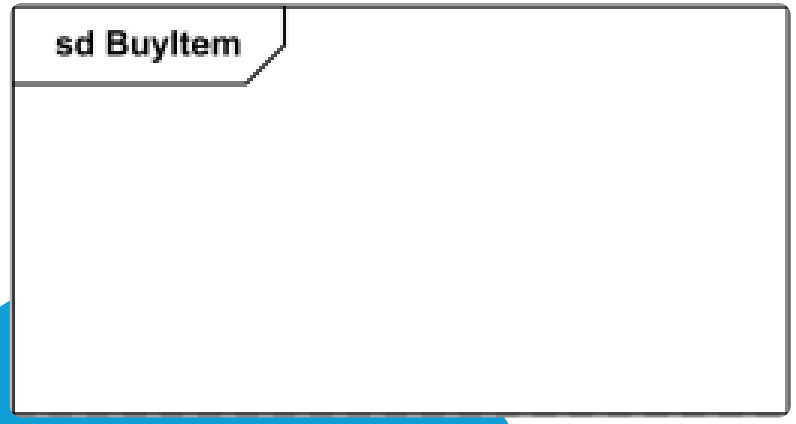
Frames

- Communication diagrams could be shown within a rectangular frame with the name in a compartment in the upper left corner.

  Provides namespace and context

  `sd` = “sequence diagram" or context identifier

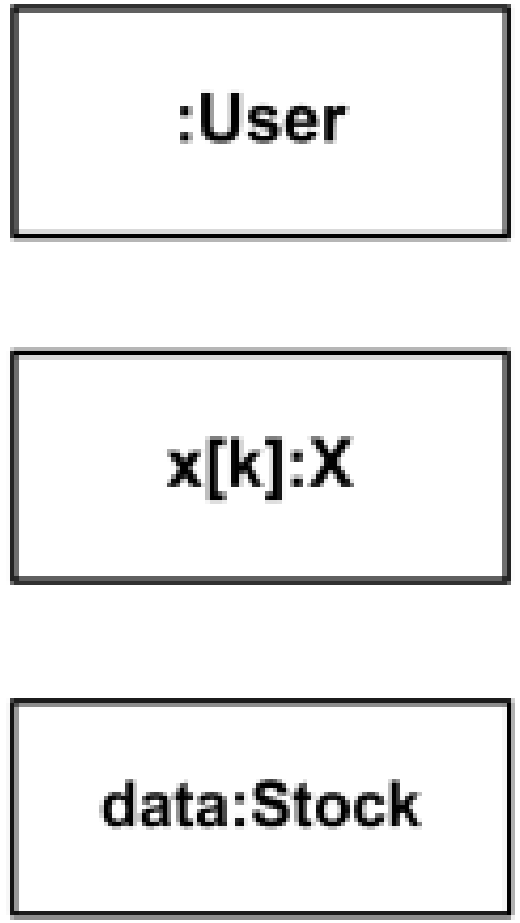
Sd ***Frame*** *for* ***Communication Diagram*** *BuyItem*



Section 17.2.4. The notation for an Interaction in a Sequence Diagram is a solid-outline rectangle. A pentagon in the upper left corner of the rectangle contains ‘sd’ followed by the Interaction name and parameters. The notation within this rectangular frame comes in several forms: Sequence Diagrams, Communication Diagrams, Interaction Overview Diagrams, and Timing Diagrams.

Lifeline

- Lifeline is a specialization of named element which represents an individual participant in the interaction.
- A lifeline is shown as a rectangle (corresponding to the head in sequence diagrams)
- The information identifying a lifeline is depicted as ObjectName[selector]: ClassName



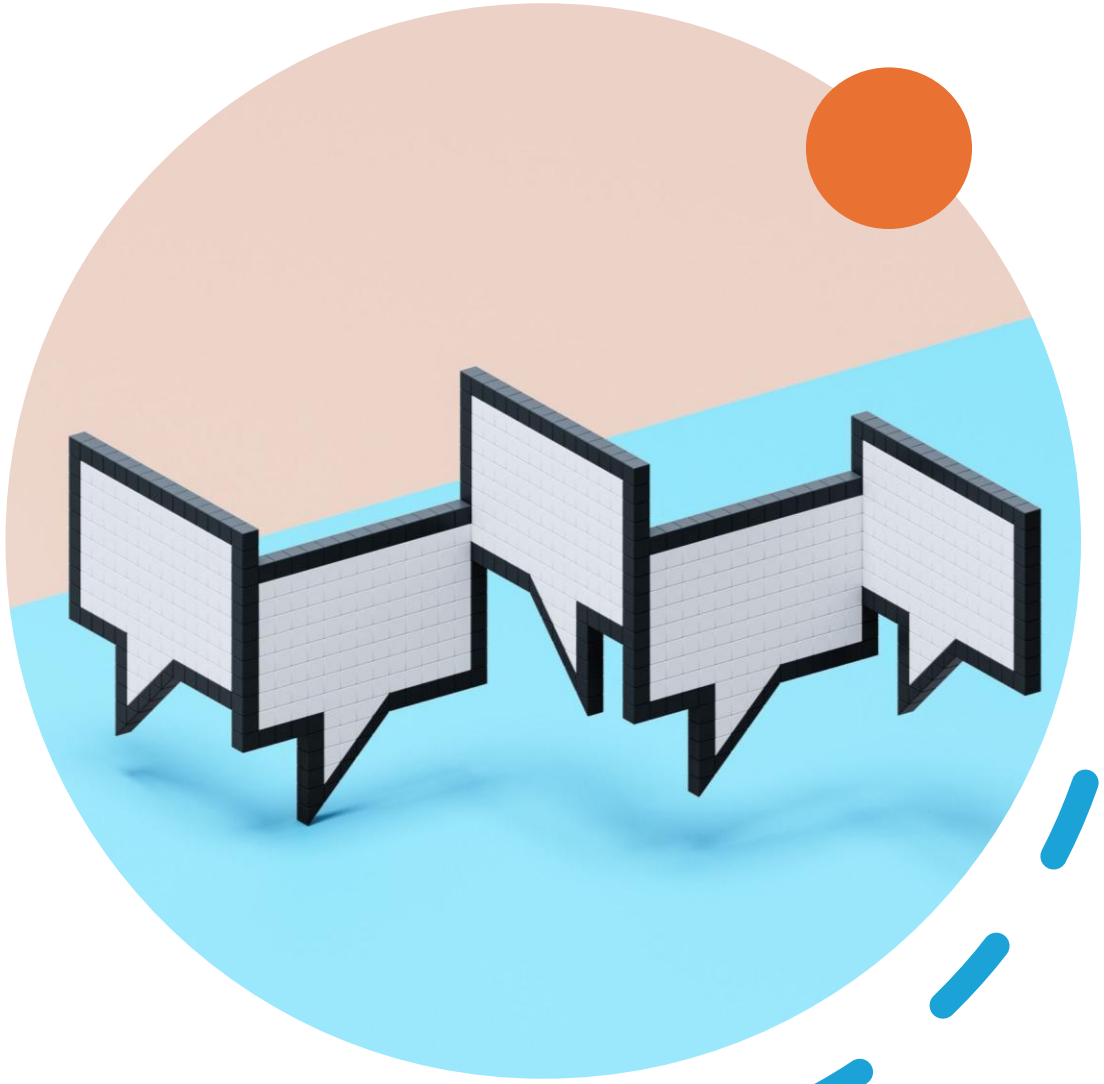
Lifeline (Sequence vs communication diagram)

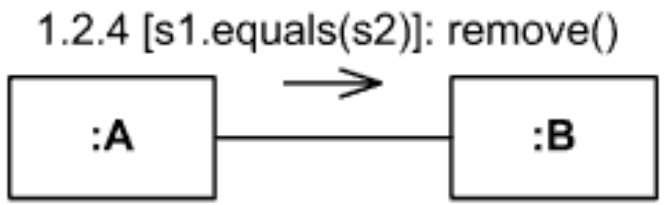
Communication: Rectangle only (no timeline)

Sequence: Rectangle with vertical lifeline below

Messages

- Message in communication diagram is shown as a line with sequence expression and arrow above the line.
- The arrow indicates direction of the communication





Instance of class A sends remove() message to instance of B if s1 is equal to s2

Sequence Expressions: The Language of Order

- General syntax

  sequence-term.sequence-term.sequence-term : messageName(parameters)

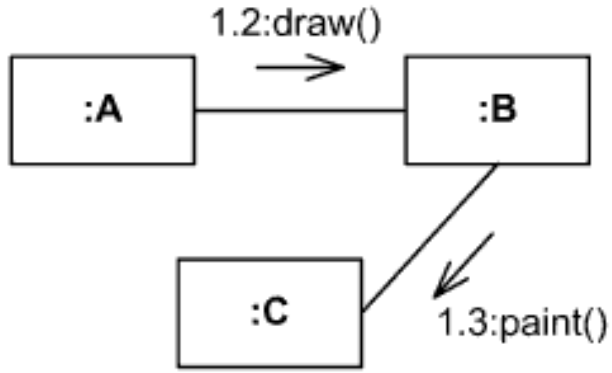
- Each sequence-term can contain:
  - Integer: Order with current level
  - Letter: Concurrent thread identifier
  - Guard: [condition] for conditional execution
  - Recurrence: * or *|| for iteration

Sequence expression

- The sequence expression is a dot separated list of sequence terms followed by a colon (“:”) and message name after that:
- Sequence-expression ::= sequence-term ‘.’…’:’ message-name
- Example: 3b.2.2:m5: sequence expression 3b.2.2 and message name m5
- Each sequence term sequence-term ::= [integer [name]] [recurrence]
- The integer represents the sequential order of the message within the next higher level of procedural calling.

Sequence Expression: Sequential Order

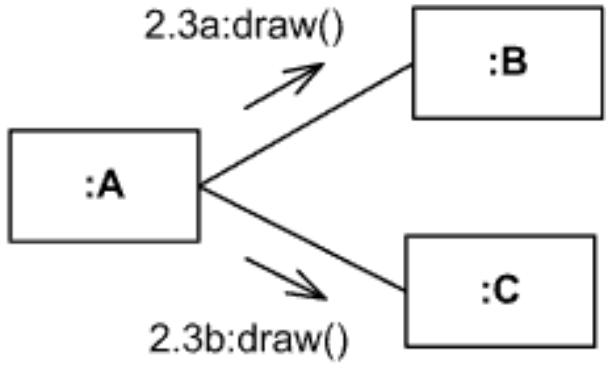
- Example
  - Message with sequence 2 follows message with sequence 1
  - 2.1 follows 2
  - 5.3 follows 5.2 within activation 5
  - 1.2.4 follows message 1.2.3 within activation 1.2



Instance of A sends draw() message to instance of B, and after that B sends paint() to C

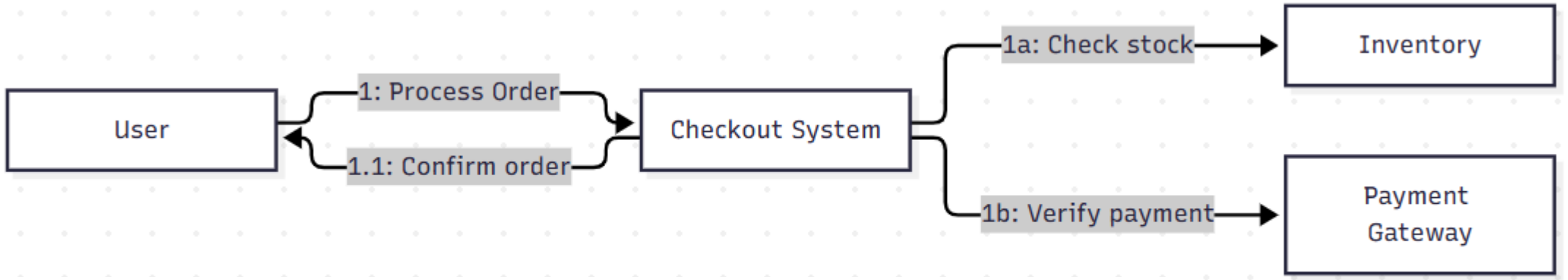
Sequence Expression: Concurrent Thread

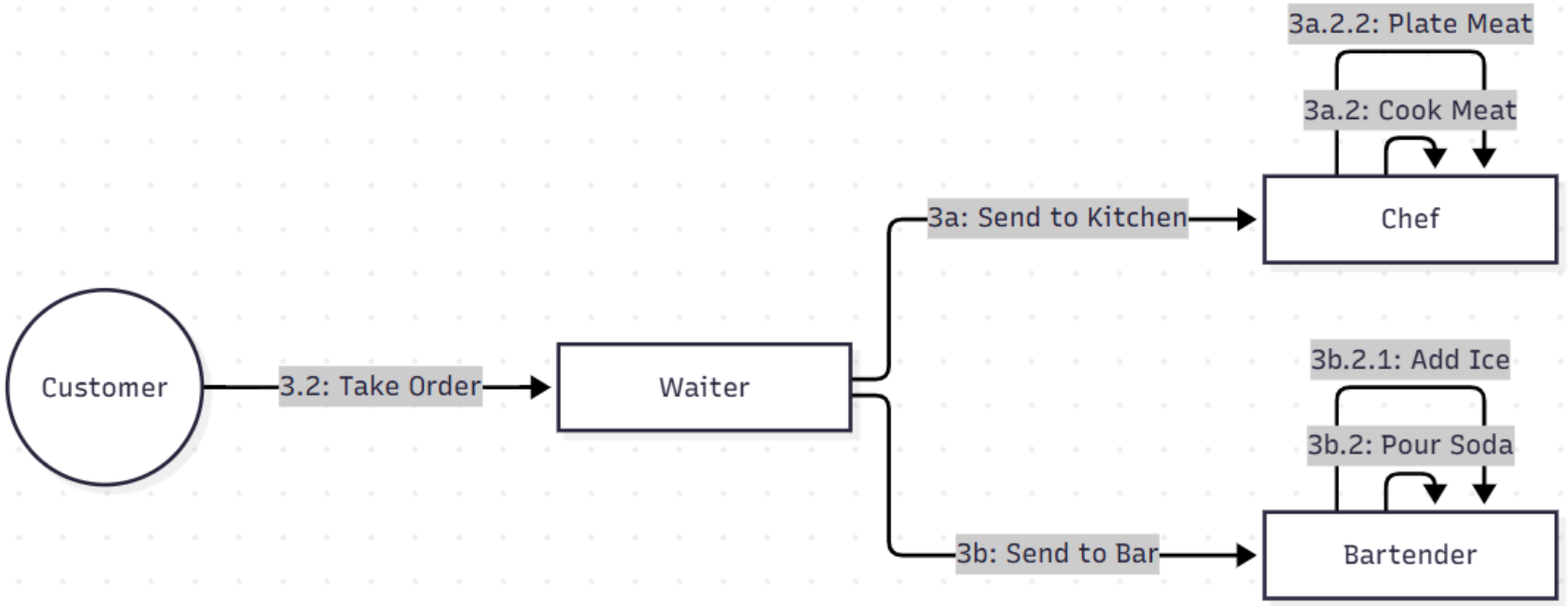
- The name represents a concurrent thread of control
- Example:
  - Messages 2.3a and 2.3b are concurrent within activation 2.3
  - 1.1 follows 1a and 1b
  - 3a.2.2 and 3b.2.1 follow 3.2



Instance of A sends draw() messages concurrently to instance of B and to instance of C

Example: 1.1 follows 1a and 1b

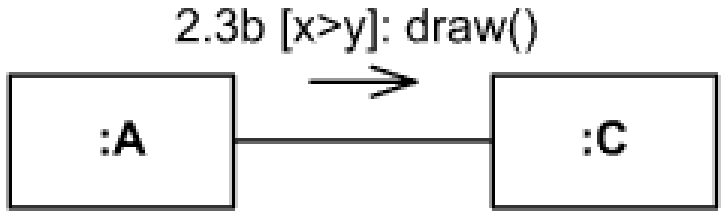




- Example: 3a.2.2 and 3b.2.1 follow 3.2

Sequence Expression: Guard

- A guard specifies condition for the message to be sent (executed) at the given nesting depth
- Example
  - 2.3b [x>y]: draw(), message draw() will be executed if x is greater than y
  - 1.1.1 [s1.equals(s2)]: remove(), message remove() will be executed if s1 equals s2



Instance of class A will send message draw() to the instance of C, if x > y

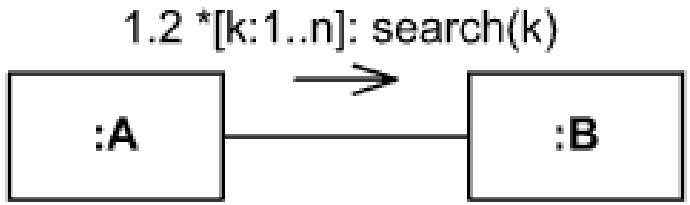
Sequence Expression: Recurrence and Iteration

- The recurrence defines conditional or iterative execution of zero or more messages that are executed depending on the specified condition
  - Recurrence ::= branch | loop, branch ::= ‘[‘guard’]’
- An iteration specifies a sequence of messages at the given nesting depth
- Notation:
  - * : Messages Executed Sequentially
  - *|| : Messages executed concurrently.

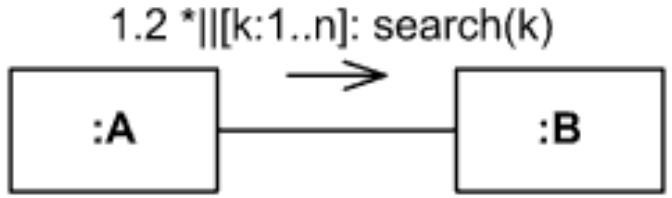
Example

- 4.2c *[i=1..12]: search(t[i]): search() will be executed 12 times, one after another
- 4.2c *||[i=1..12]: search(t[i]): 12 search() messages will be sent concurrently
- 2.2 *: notify(): message notify() will be repeated some unspecified number of times.

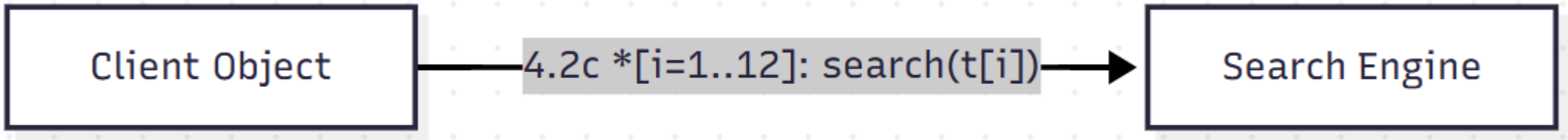
Examples

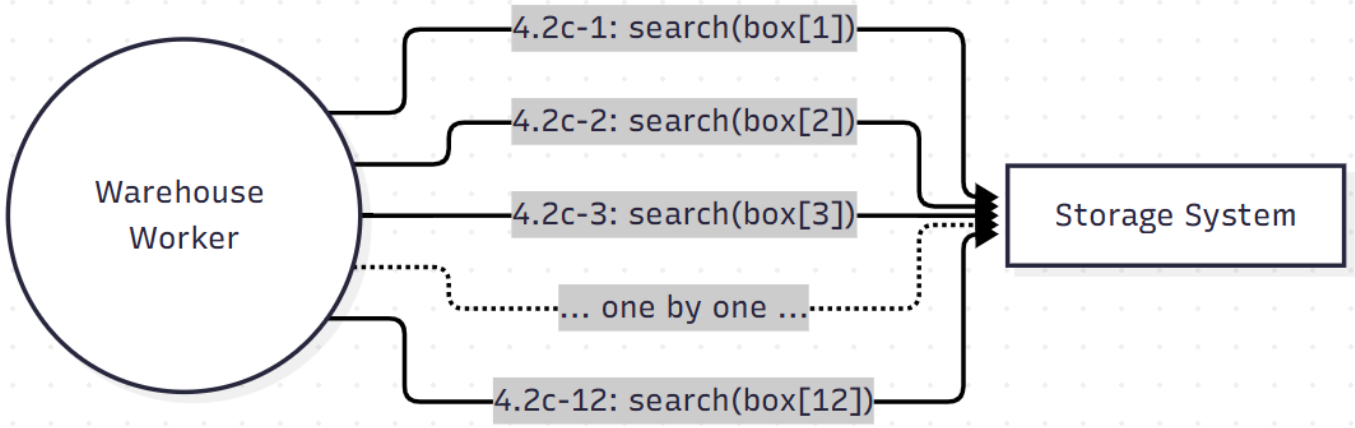


Instance of class A will send search() message to instance of B n times, one by one

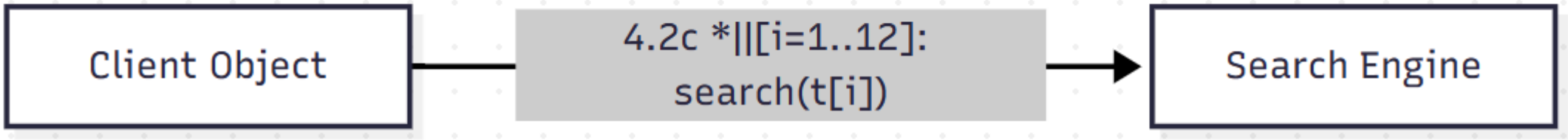


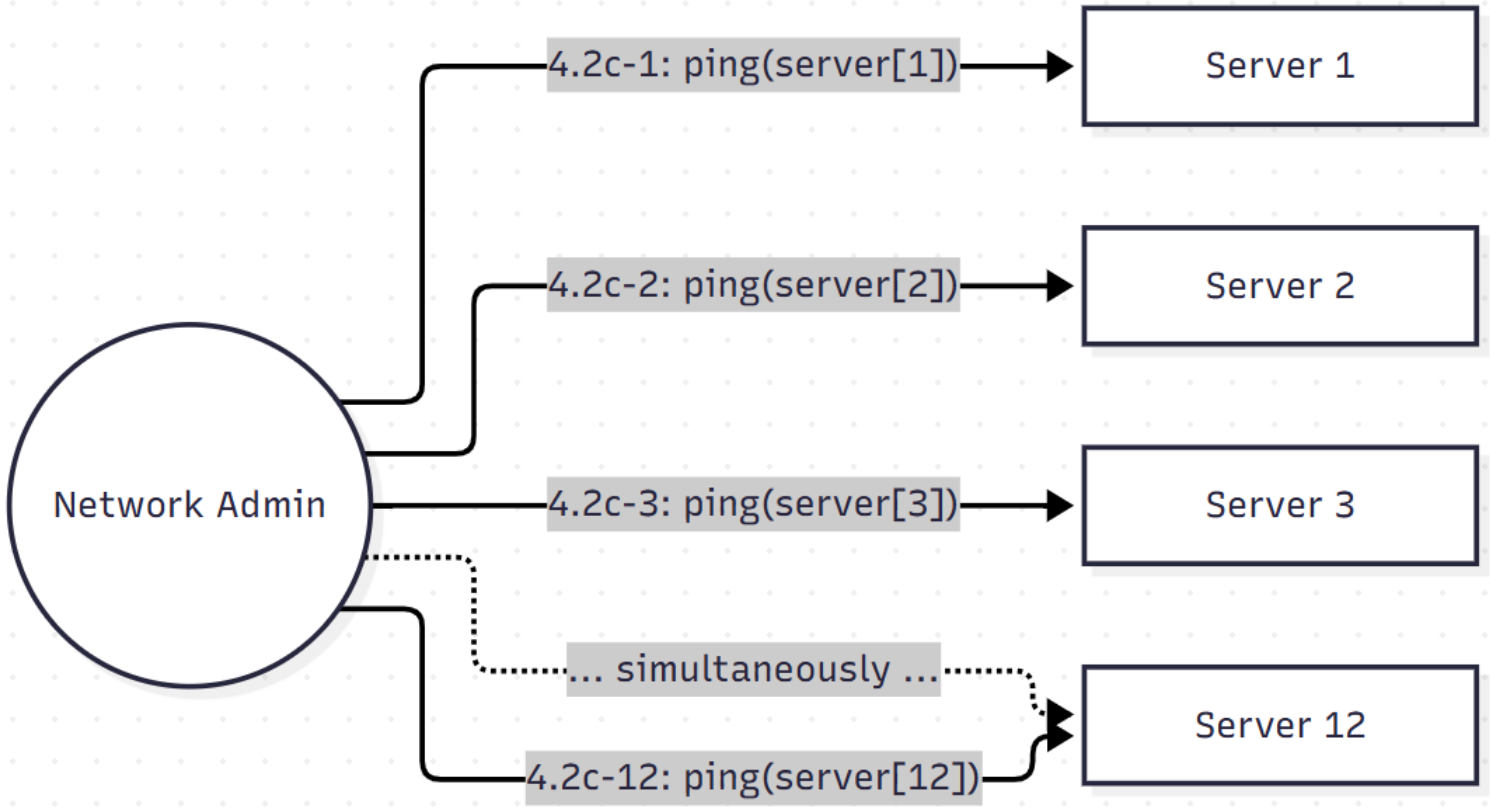
Instance of class A will send n concurrent search() messages to instance of B



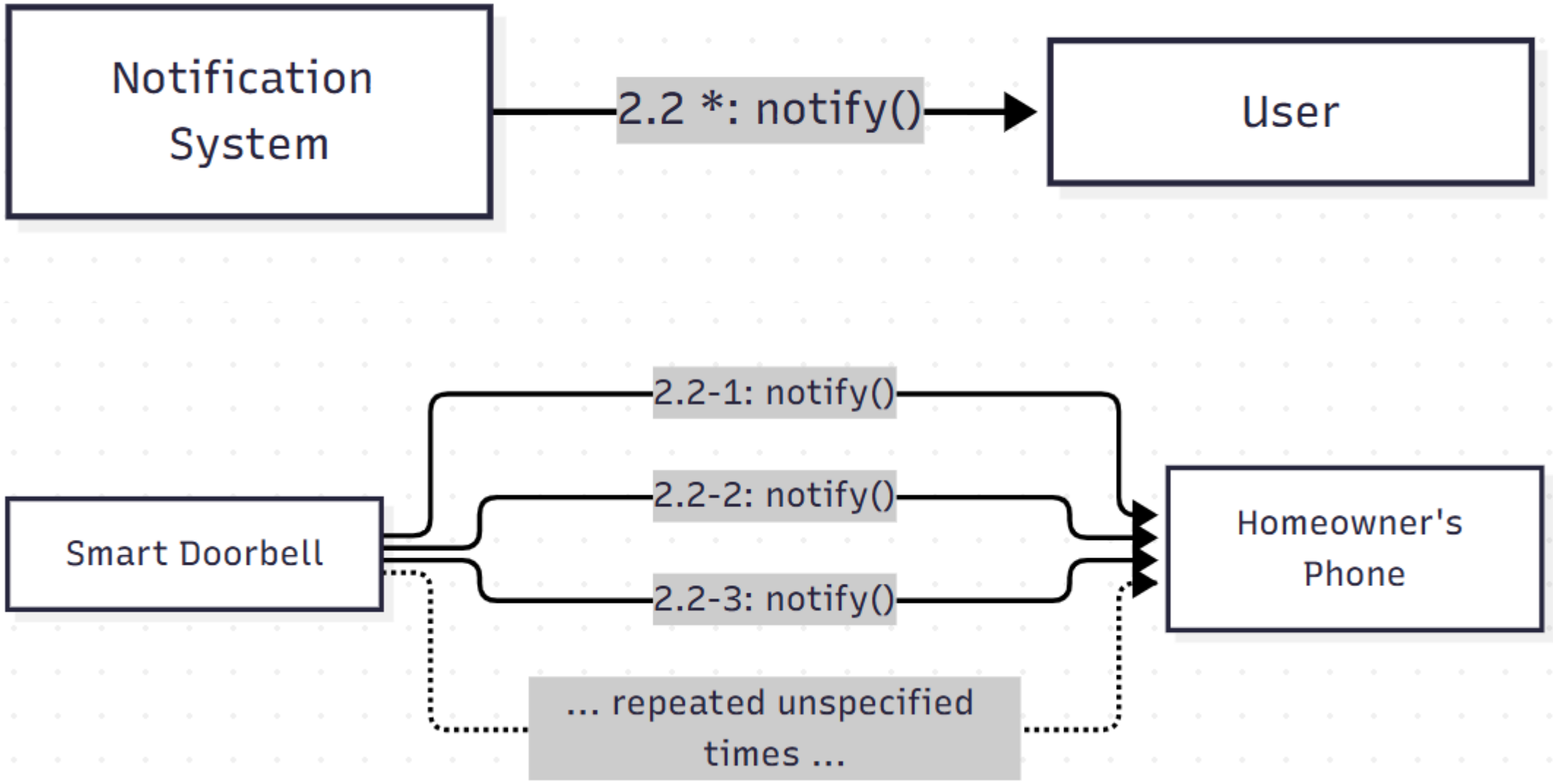


4.2c *[i=1..12]: search(t[i]): search() will be executed 12 times, one after another



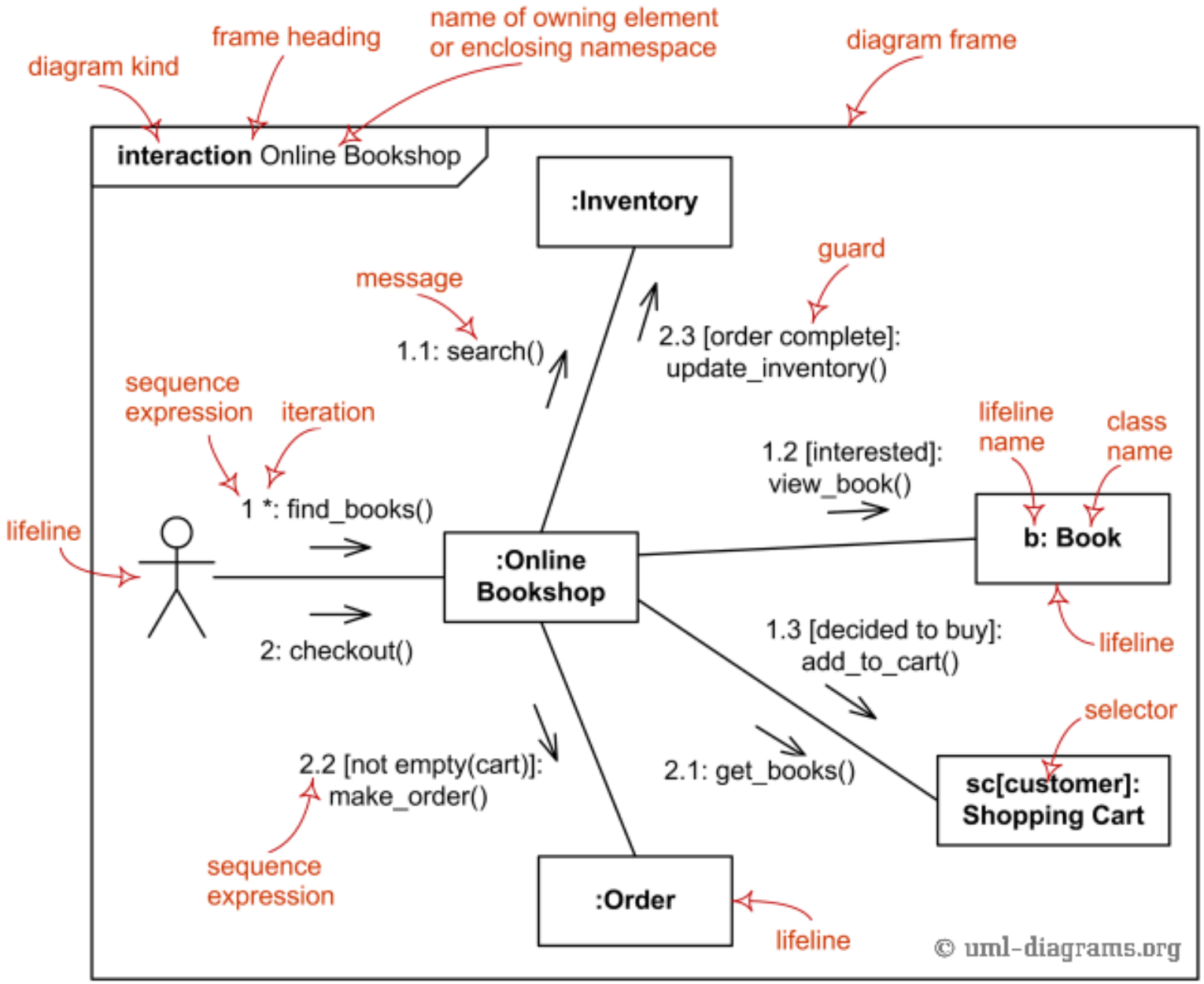


4.2c *||[i=1..12]: search(t[i]): 12 search() messages will be sent concurrently

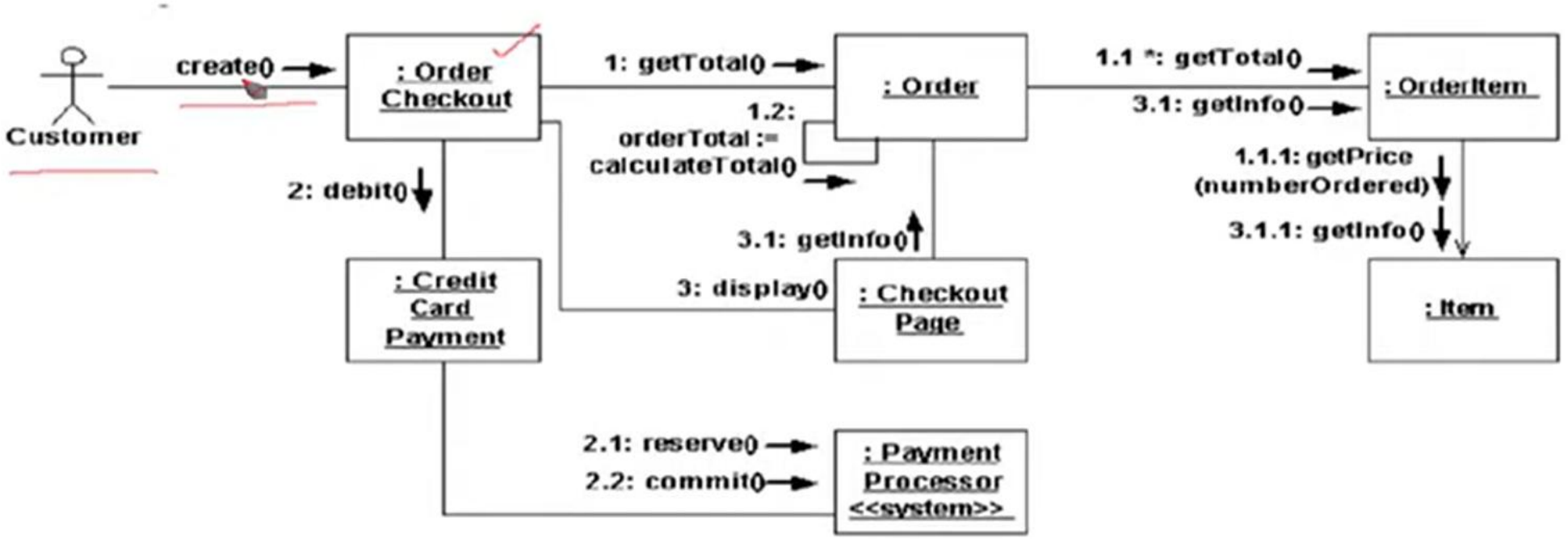


2.2 *: notify(): message notify() will be repeated some unspecified number of times.

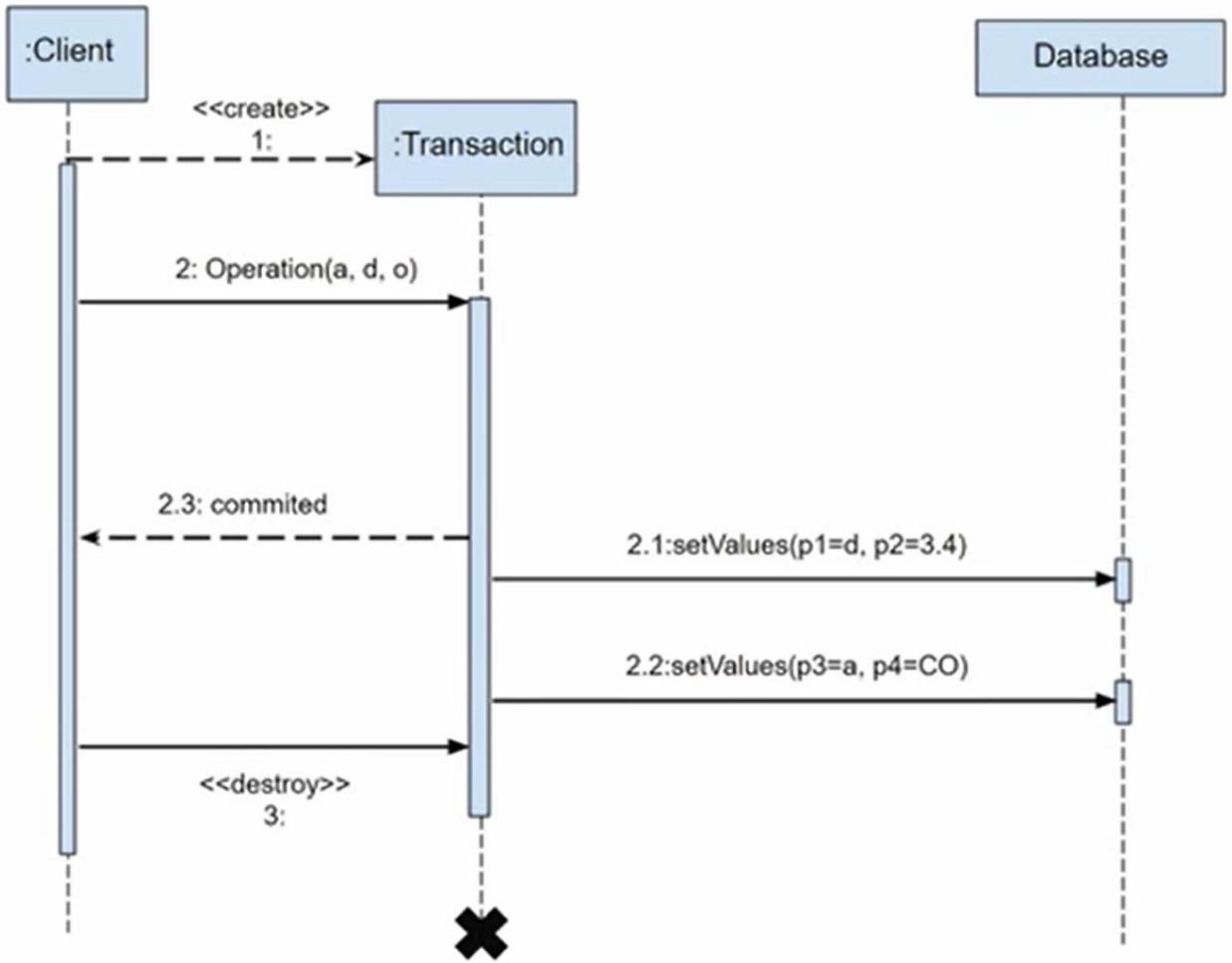
Annotated Online Book shop

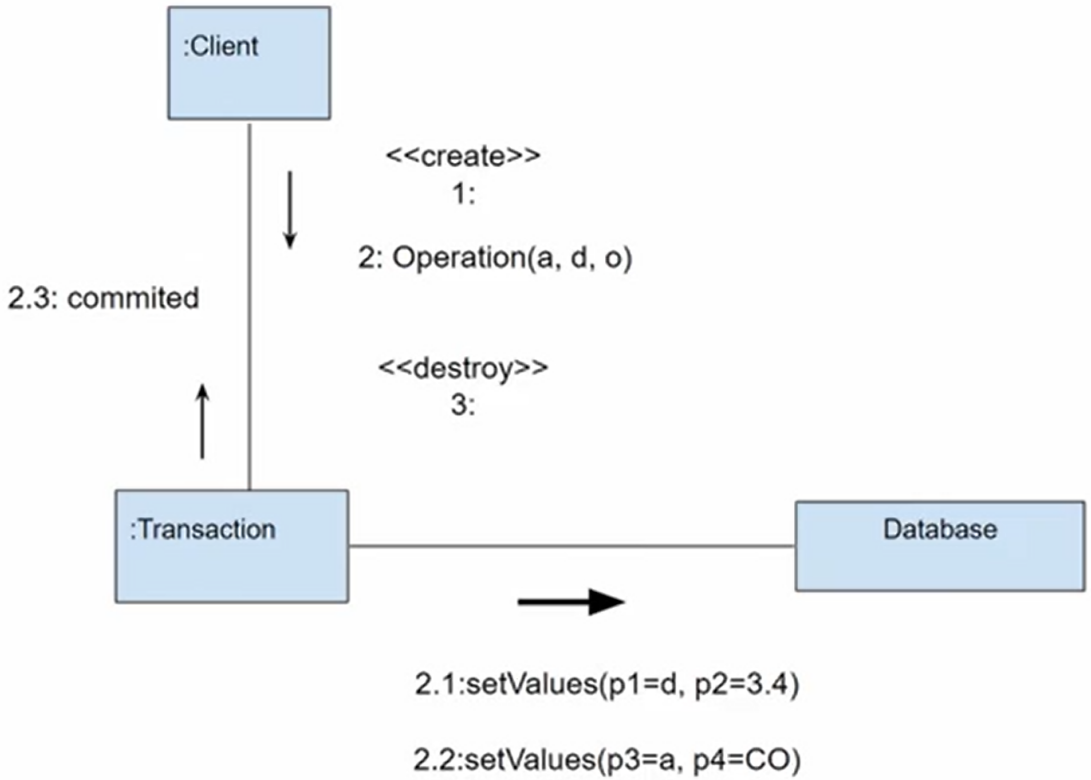


Order Management

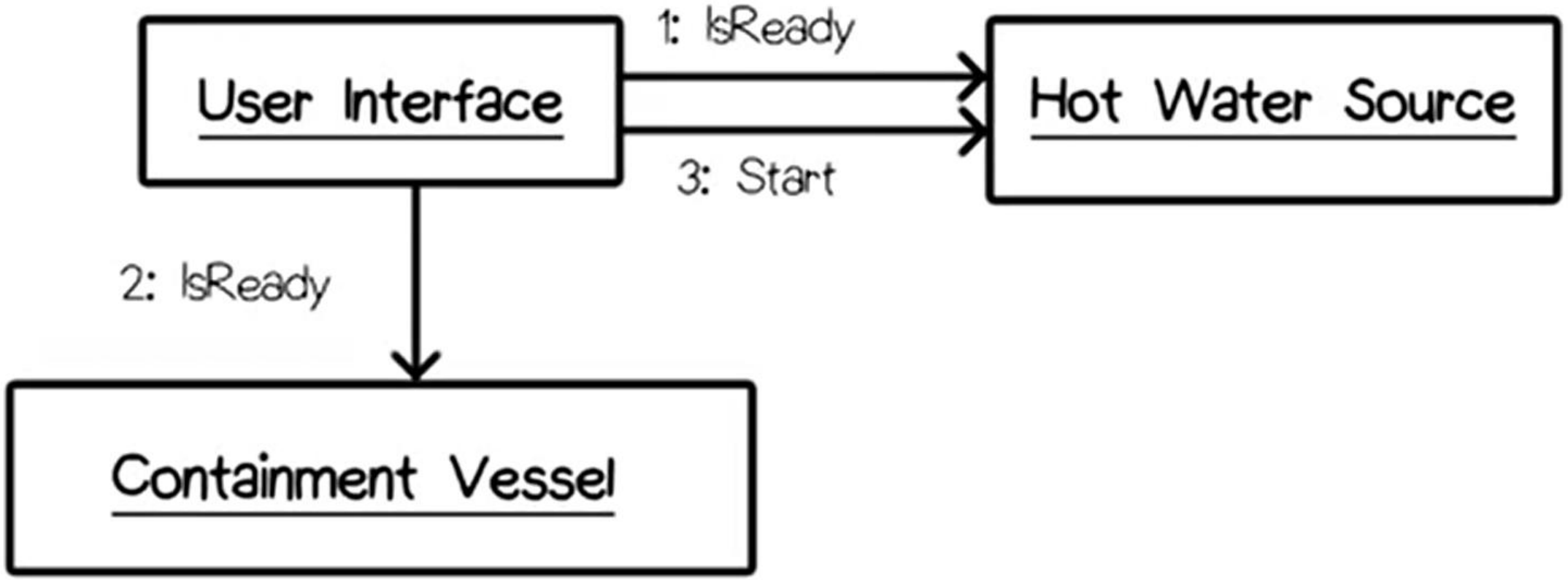


Sequence vs Communication Diagrams

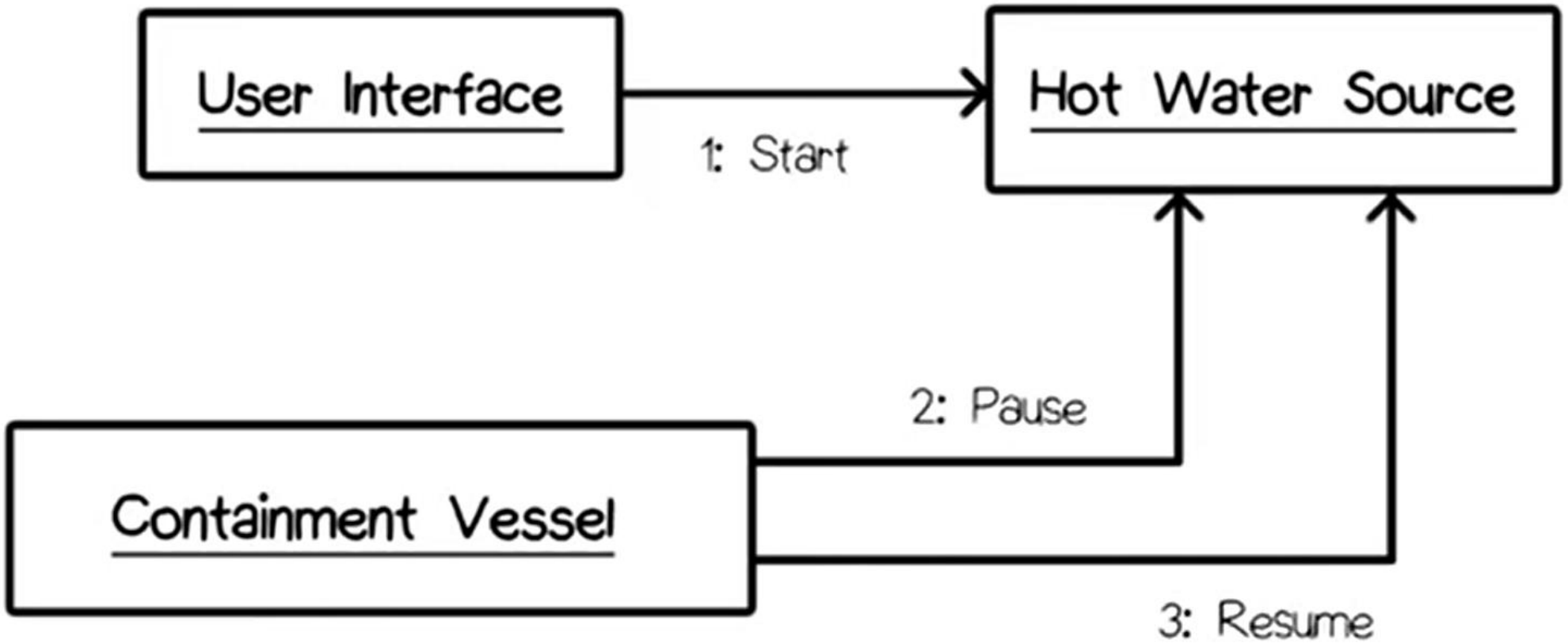




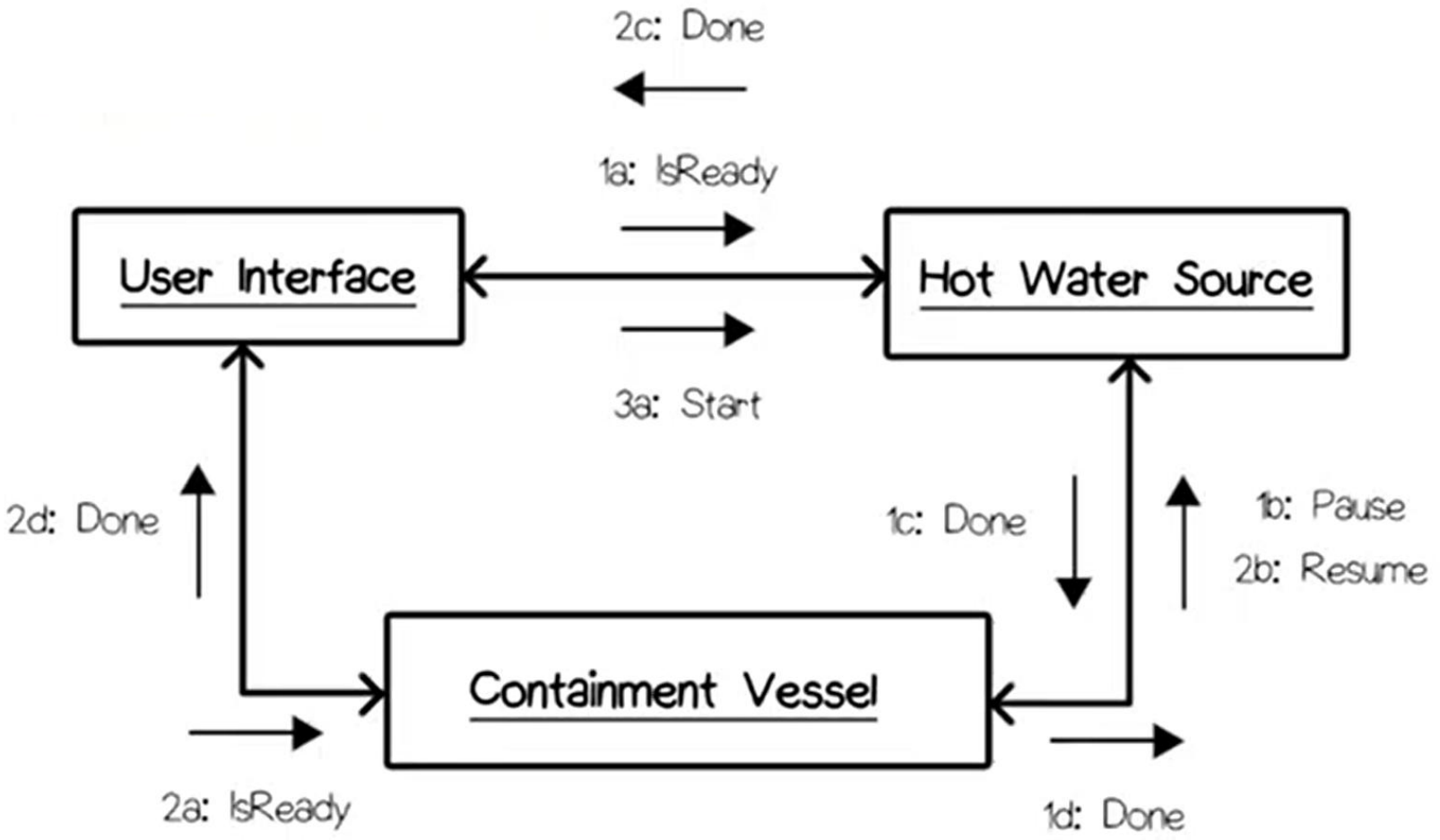
Discussion: Use Case 1



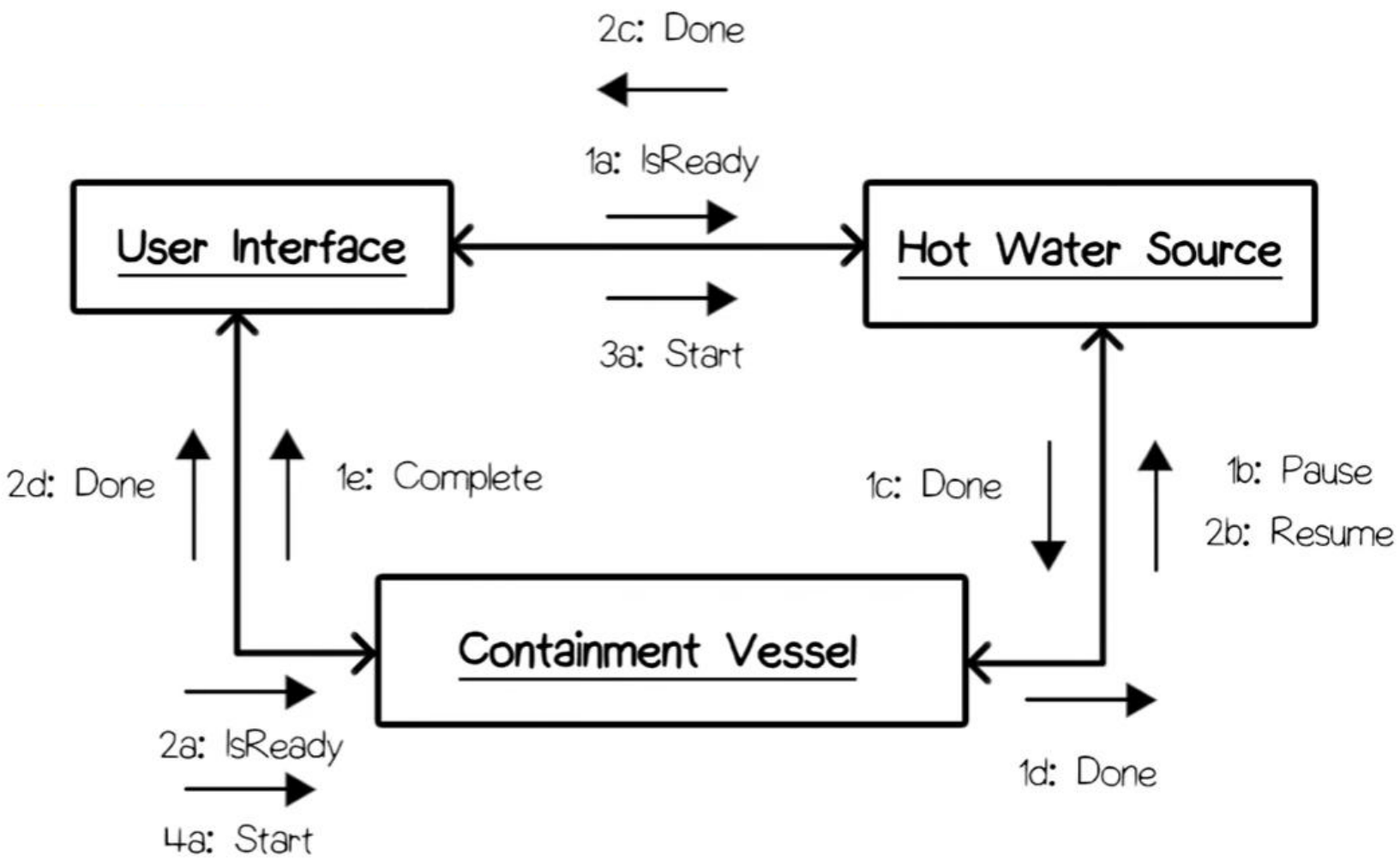
Discussion – Use Case 2



Discussion – Use Case 3



Discussion – Use Case 4



Common Mistakes and Best Practices

❌ Common Errors to Avoid

1. Incorrect Sequence Numbering

1. Wrong: `1, 2, 1.1, 2.1` (nested after parent completed)
2. Right: `1, 1.1, 2, 2.1` (nested during parent execution)

2. Misusing Concurrent Threads

1. Wrong: `1a, 2b` (different nesting levels)
2. Right: `1.1a, 1.1b` (same level for concurrency)

3. Guard Placement

1. Wrong: `[condition] 2.1: message()`
2. Right: `2.1 [condition]: message()`

4. Duplicate Sequence Numbers

1. Each message must have unique sequence expression

5. Forgetting Return Messages

1. While optional, return messages help show control flow
2. Notation: `1.1r: result` or use dashed arrow

✅ Best Practices

1. Start Simple: Begin with main flow, add complexity incrementally

2. Use Guards Sparingly: Too many conditions make diagrams hard to read

3. Group Related Objects: Spatial layout should reflect logical grouping

4. Be Consistent: Stick to one numbering style throughout

5. Document Assumptions: Add notes for complex guard conditions

6. Consider Your Audience: Adjust detail level for stakeholders vs. developers

Key Takeaways

- Communication Diagrams emphasize architecture over timing
- Sequence expressions use dot notation to show nesting depth
- Letters indicate concurrent threads at the same level
- Guards enable conditional execution
- Asterisks (* and *||) control iteration behavior
- Choose communication diagrams when object relationships matter more than precise timing

Additional Resources

- UML 2.5 Specification: Communication Diagrams
- “UML Distilled” by Martin Fowler
- Online Tool: draw.io, Lucidchart, PlantUML for practice
- Good video tutorials: https://www.youtube.com/watch?v=Xe0VZ2Fw7FQ
- https://www.youtube.com/watch?v=TL4ABTx_RtE

